



## Corso di Industrial and Automotive Real-Time Networks

# Automotive Real-Time Switched Ethernet Network Simulation

Federico Calabrese, 1000085461, Stefano Caramagno, 1000038343

### I. INTRODUZIONE

Questo lavoro presenta la **simulazione** di una rete **Switched Ethernet a 10 Gb/s** per un'applicazione **automotive** caratterizzata da traffico *cross-domain*.

La topologia di rete impiega **18 end-node** e **4 switch** Ethernet e veicola dati afferenti a due domini funzionali: **ADAS** (*Advanced Driver Assistance Systems*) e **Multimedia/Infotainment**.

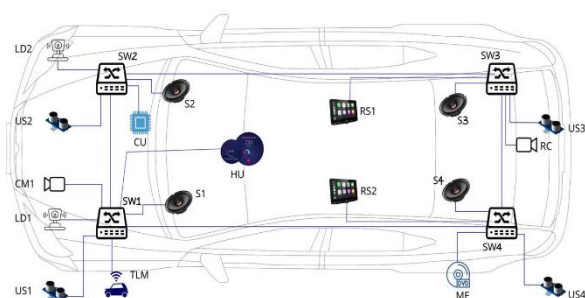


Figura 1: Schema scenario implementato

#### A. Descrizione flussi

I flussi scambiati dai vari end-node sono riportati nella tabella I.

Ogni nodo esegue un numero di applicazioni pari alla somma dei flussi in trasmissione e in ricezione.

È stata implementata una sola coda di trasmissione per porta Ethernet senza limiti di dimensioni. Sono state valutate le seguenti metriche:

- Frame-Loss Ratio, per ogni flusso.
- End-to-end delay, per ogni flusso;
- Numero massimo di frame in coda, per ogni porta.

### II. SCENARIO

Lo scenario di riferimento, illustrato in **Figura 1**, è articolato in due distinti domini funzionali automotive:

- **ADAS** (*Advanced Driver Assistance Systems*);
- **Multimedia/Infotainment**.

Nel dominio **ADAS** sono previsti: due videocamere, una anteriore (**CM1**) e una posteriore (**RC**), quattro sensori ad ultrasuoni (**US1–US4**) e due sensori **LiDAR** (**LD1, LD2**). Le due videocamere trasmettono i flussi video alla **Head Unit (HU)**, che fornisce assistenza visiva al conducente. I sensori ad ultrasuoni e i LiDAR inviano i dati campionati a una **ECU** denominata **Control Unit (CU)**, responsabile dell'elaborazione e dell'inoltro degli avvisi al conducente tramite la HU. Nello scenario considerato, la videocamera anteriore trasmette messaggi di **178 500 byte** con **periodo pari a 16,66 ms**; la videocamera posteriore, invece, utilizzata durante le manovre, trasmette messaggi di **178 000 byte** con **periodo pari a 33,33 ms**.

Il dominio **Multimedia/Infotainment** comprende un sistema multimediale composto da quattro altoparlanti (**S1–S4**) e due **Rear Seat Entertainment** (**RS1, RS2**). Il nodo **Multimedia Entertainment (ME)** trasmette un flusso video verso ciascun dispositivo **RS** (**RS1, RS2**) e un flusso audio verso ciascun altoparlante (**S1–S4**), con **periodo di 250  $\mu$ s**. Lo scenario include, inoltre, un sistema di **telematica** che invia informazioni alla HU e alla CU (ad esempio: avvisi sul traffico, dati GPS, mappe).

I sistemi **Multimedia/Infotainment** e di **telematica** sono **real-time**, ma **non safety-critical**.

#### A. Priorità assegnate

Nella configurazione da noi implementata viene utilizzata una strategia di schedulazione basata su **Strict Priority Selection**.

| Flow Id | Src                | Dst         | Periodo     | Deadline Relativa | Payload     | Burst |
|---------|--------------------|-------------|-------------|-------------------|-------------|-------|
| 1       | LD1, LD2           | CU          | 1.4 ms      | 1.4 ms            | 1300 byte   | 2     |
| 2       | ME                 | S1,S2,S3,S4 | 250 $\mu$ s | 250 $\mu$ s       | 80 byte     | 1     |
| 3       | US1, US2, US3, US4 | CU          | 100 ms      | 100 ms            | 188 byte    | 1     |
| 4       | CU                 | HU          | 10 ms       | 10 ms             | 10500 byte  | 12    |
| 5       | CM1                | HU          | 16.66 ms    | 16.66 ms          | 178500 byte | 255   |
| 6       | ME                 | RS1, RS2    | 33.33 ms    | 33.33 ms          | 178500 byte | 357   |
| 7       | TLM                | HU, CU      | 625 $\mu$ s | 625 $\mu$ s       | 600 byte    | 1     |
| 8       | RC                 | HU          | 33.33 ms    | 33.33 ms          | 178000 byte | 356   |

Per l'assegnazione delle priorità è stata sfruttata la deadline relativa ( $D_i$ ), secondo la relazione:

$$P_i = \frac{1}{D_i}$$

Nella configurazione da noi implementata viene usata una strategia di schedulazione basata su Strict Priority Selection.

Tali priorità sono codificate all'interno delle frame Ethernet IEEE 802.1Q. Le priorità ottenute sono riportate in Tabella II.

| Src                | Dst         | Deadline Relativa | Priorità |
|--------------------|-------------|-------------------|----------|
| LD1, LD2           | CU          | 1.4 ms            | 5        |
| ME                 | S1,S2,S3,S4 | 250 $\mu$ s       | 7        |
| US1, US2, US3, US4 | CU          | 100 ms            | 1        |
| CU                 | HU          | 10 ms             | 4        |
| CM1                | HU          | 16.66 ms          | 3        |
| ME                 | RS1, RS2    | 33.33 ms          | 2        |
| TLM                | HU, CU      | 625 $\mu$ s       | 1        |
| RC                 | HU          | 33.33 ms          | 2        |

Tabella 2: Caratteristiche dei flussi

## B. Dettagli implementativi

La rete implementata interconnette 18 end-node con l'ausilio di 4 switch (SW1, SW2, SW3, SW4).

### 1) Configurazione del Data Rate (10 Gb/s) e del Bit Error Rate (BER)

Per garantire una **data rate** pari a **10 Gb/s** su tutti i collegamenti, sono stati modificati due elementi del progetto **OMNeT++**. In primo luogo, all'interno del modulo *EtherMAC.ned* è stato impostato un valore di **default** della data rate pari a **10 Gb/s**, così da evitare incongruenze e colli di bottiglia nella configurazione dello scenario. In secondo luogo, è stato introdotto un **compound channel** dedicato (*CustomChannel.ned*), che estende il canale a data rate del framework (*ned.DatarateChannel*) esponendo esplicitamente due parametri: *dataRate* e *ber*.

Nel canale, di **default** il *dataRate* è fissato a **10 Gb/s**, mentre il *ber* è impostato a  $10^{-8}$  per soddisfare il **Caso 1**; per il **Caso 2** la presenza di BER può essere disabilitata (commentando/attivando il parametro e ricompilando), così da simulare un canale ideale.

Il file in cui è descritta la topologia di scenario (ovvero *Scenario2.ned*) utilizza *CustomChannel* per tutte le interconnessioni **host-switch** e **switch-switch**, con notazione esplicita delle porte (esempio schematico: *sw1.channelIn[0]  $\leftarrow$  CustomChannel  $\leftarrow$  sw2.channelOut[0]*). Questa impostazione rispetta i vincoli della **Variante 1** (10 Gb/s; BER nel Caso 1 / assenza di BER nel Caso 2) indicati in consegna.

### 2) Ridondanza delle trasmissioni e controllo di inoltro tramite VLAN

Per realizzare la **ridondanza per singolo flusso** lungo **percorsi distinti** tra coppie di nodi, si è deciso di utilizzare un filtraggio **tramite VLAN**, in modo che, per ogni porta, sia possibile stabilire l'insieme dei flussi ammessi in **uscita**. Trattando separatamente i flussi elementari, si ottengono **17** relazioni punto-punto e, di conseguenza, **17 VLAN ID** dedicati.

Per limitare la complessità e contenere la propagazione dell'errore (poiché un percorso esterno più lungo aumenta il numero di collegamenti attraversati e quindi l'esposizione al **BER** cumulato), si è stabilito di **abilitare la ridondanza solo quando tra due host intercorrono almeno due switch**. Qualora i due host insistano sul **medesimo switch**, sarà presente un *vlanId* che specifica la **porta** su cui trasmettere il pacchetto, ma il flusso **non** verrà ridondato. Questo criterio mantiene la robustezza senza penalizzare eccessivamente latenza e affidabilità percepite.

Dal punto di vista implementativo, in *EtherMAC.cc* è stata introdotta una **verifica di ammissione in trasmissione** (funzione *vlanFilter()*): se il pacchetto in uscita presenta un **VLAN ID abilitato** su quella porta, il frame viene inserito in coda di trasmissione; in caso contrario, viene **scartato**. In tal modo, la ridondanza è realizzata **a priori** tramite la progettazione delle VLAN.

L'introduzione di percorsi ridondanti comporta il problema della **uplicazione** dei pacchetti presso il ricevente. Per evitare conteggi multipli (ad es. sulla **FLR**) e garantire tracciamenti consistenti, in *PeriodicTrafficGenerator.cc* è stato implementato un **meccanismo di scarto dei duplicati**. Si mantiene una struttura indicizzata da una **chiave composta** (**pktNumer**, **genTime**): se arriva un frame con **coppia già ricevuta**, il pacchetto viene scartato come duplicato; se giunge un frame con *pktNumber* inedito o già visto ma con *genTime* **maggiore** di quello registrato, la mappa viene aggiornata. Il successivo posizionamento delle statistiche garantisce che **solo l'istanza più recente** contribuisca alle metriche.

### 3) Mappatura con Strict Priority e frammentazione delle frame

#### Selezione Strict Priority su coda di trasmissione unica

In *EtherMAC.cc* la coda di trasmissione (*txQueue*) è implementata come *cPacketQueue* con **comparatore personalizzato** basato sul campo **PCP — Priority Code Point**. La *cPacketQueue* può essere inizializzata passando, oltre al nome della coda, anche un **oggetto comparatore** (cfr. documentazione: “*Base class for object comparators, used by cQueue for priority queuing.*”). Il comparatore, implementato come *pcpComparator*, ridefinisce l'operazione di *pop* in modo da **estrarre sempre** il frame con **PCP più elevato**, implementando una **Strict Priority Selection** su un'unica coda logica. Tale modellazione privilegia la semplicità e riproduce, dal punto di vista funzionale, il comportamento della **Strict Priority Selection**: i flussi a priorità maggiore prevaricano sistematicamente quelli a priorità inferiore, riducendone la latenza.

Le **priorità di flusso** sono derivate **inversamente** dalla **deadline** (e quindi, nel nostro caso anche dal periodo): a **deadline relative più brevi** corrisponde una priorità più alta. Sono stati definiti **7 livelli** ordinati (da **1** a **7**).

#### Frammentazione delle frame

La **frammentazione** è implementata nel generatore di traffico, ossia in *PeriodicTrafficGenerator.cc*, più nello specifico dalla funzione *transmitPacket*. Per ogni flusso si imposta un parametro *burstSize* (numero di frammenti per pacchetto applicativo). Data la dimensione iniziale del payload, la dimensione del frammento è calcolata come:

$$fragSize = \left\lceil \frac{payloadSize}{burstSize} \right\rceil$$

Si emettono quindi *burstSize* frame; per l'ultimo frammento si applica un **clamp** con *min* per non superare i byte residui.

#### Politica di assegnazione di *burstSize*:

- **Priorità 7 e 6:** *burstSize* = 1 (payload ridotto; non si otterrebbe alcun beneficio ulteriore dalla sua suddivisione).
- **Priorità 5:** *burstSize* = 2 (nonostante il payload sia inferiore all'MTU di Ethernet, si è scelto di frammentare per ottenere un migliore bilanciamento tra latenza e rispetto delle priorità).
- **Priorità 4:** *burstSize* = 12 (frammentazione necessaria; frammenti inferiori alla MTU; bilanciamento tra latenza e priorità).
- **Priorità 2:** *burstSize* = 356 o 357 (frammentazione necessaria dato il **payload**).
- **Priorità 1:** *burstSize* = 1 (frammentazione non necessaria, alla luce del payload di **188 B**; sebbene la priorità sia bassa, non è stato ritenuto utile frammentare ulteriormente).

Questo schema realizza una **preferenzialità temporale** per i flussi *time-critical*, coerente con la Strict Priority della coda di trasmissione, e sposta il costo della frammentazione verso i flussi a **bassa priorità**.

### 4) Implementazione delle valutazioni: FLR, end-to-end delay (frame e burst), massima lunghezza della coda di trasmissione

La raccolta e la registrazione delle metriche si basano sul meccanismo di **segnali** e **statistiche** di OMNeT++. Ogni grandezza monitorata è associata a un **signal** (dichiarato nei file *.ned* e referenziato nei file *.cc/.h*) ed è emessa in simulazione tramite *emit()*, che produce una coppia (**value**, **simTime**) registrata nei risultati. In questo modo, i **vector** generati consentono sia il **plotting** temporale sia l'applicazione di **aggregatori** (ad es. il massimo) definiti a livello di statistica NED.

- **Frame Loss Ratio (FLR).** Calcolata nel generatore periodico come rapporto:

$$FLR = \frac{\#frame\ scartate}{\#frame\ trasmesse}$$

ed emessa come segnale durante la simulazione, in accordo con la consegna.

- **End-to-End Delay (E2E) per frame.** Per ciascun frammento si computa:

$$E2Eframe = RxTime - GenTime$$

ed il valore viene emesso via *emit()* all'arrivo del frame.

- **End-to-End Burst Delay.** Per rappresentare il ritardo dell'intero pacchetto applicativo si considera **solo l'ultimo frammento** (quello con indice pari a *burstSize*) e si calcola:

$$E2Eburst = RxTime(last) - GenTime(last)$$

emettendo il risultato come segnale dedicato.

- **Massima lunghezza della coda di trasmissione.** In *EthernMAC.cc* viene emesso un segnale ogni volta che un pacchetto viene inserito in coda di trasmissione. Nel NED si definisce una **statistica** che usa questo segnale come sorgente e registra il **massimo** (*record = max*) sull'intera esecuzione. La metrica così ottenuta corrisponde al “**numero massimo di frame in coda**” previsto in consegna.

Grazie allo **scarto dei duplicati** lato ricevente, le metriche (FLR, ritardi E2E frame/burst, saturazione coda) risultano coerenti anche in presenza di **percorsi ridondanti**, evitando conteggi doppi e fornendo una stima fedele del comportamento del sistema nei due casi di canale considerati (BER  $10^{-8}$  vs. assenza di BER).

### C. Metriche valutate

Le metriche valutate sono:

- Frame-Loss Ratio, per ogni flusso.
- End-to-end delay, per ogni flusso;
- Numero di frame in coda, per ogni porta;

L'FLR è stato calcolato come rapporto tra il numero di frame scartate rispetto a quelle trasmesse, come visto prima:

$$FLR = \frac{\#frame\ scartate}{\#frame\ trasmesse}$$

Il numero di frame scartate è stato calcolato come la differenza tra il numero di frame trasmesse (*frameTX*), codificato all'interno del pacchetto, e il numero di frame ricevute (*frameRX*).

$$FLR = \frac{frameTX - frameRX}{frameTX} = 1 - \frac{frameRX}{frameTX}$$

## III. RISULTATI SIMULAZIONI

### 1) Frame-Loss Ratio (FLR)

La simulazione è stata eseguita per **10 s** in due configurazioni:

- canale **senza BER**;
- canale con **BER** pari a  $10^{-8}$ .

In entrambi i casi, le restanti metriche considerate nel lavoro (*end-to-end delay*); massimo numero di frame in coda **non mostrano variazioni apprezzabili** al variare della BER nell'orizzonte temporale considerato; la dinamica globale del sistema, pertanto, risulta **sostanzialmente invariata** sotto questo profilo.

#### Caso senza BER

In assenza di errori di bit, la **FLR** risulta **praticamente nulla per tutti i flussi**. Il comportamento è coerente con l'aspettativa teorica: non essendo introdotti errori dal canale, **EthernMAC.cc** non effettua scarti e, di conseguenza, non si osservano perdite.

#### Caso con BER $10^{-8}$

Con l'introduzione di una BER pari a  $10^{-8}$ , la FLR diviene **non nulla** su **alcuni** flussi entro i **10 s** di osservazione (su altri può rimanere a zero nell'intervallo considerato). Tale evidenza è **consistente** con l'orizzonte limitato della simulazione: prolungando la durata, **tutti i flussi** mostrerebbero prima o poi perdite, con un corrispondente andamento della FLR nel tempo.

#### Flussi con percorso ridondante

Per i flussi che dispongono di **ridondanza di percorso** (impostata a livello VLAN), l'andamento tipico della FLR è quello mostrato, ad esempio, dai flussi **RC** → **HU**, **CU** → **HU** e **ME** → **RS1**:

- si osservano **incrementi puntuali** (picchi) in corrispondenza della perdita di uno o più frame;
- **escludendo tali outlier**, la FLR **decresce** progressivamente nel tempo, poiché il numero cumulato di frame ricevuti con successo cresce molto più rapidamente del numero di frame persi. In altri termini, la ridondanza mitiga l'impatto delle perdite isolandole in transitori brevi e favorendo una **rapida “riconvergenza”** della FLR verso valori molto bassi.

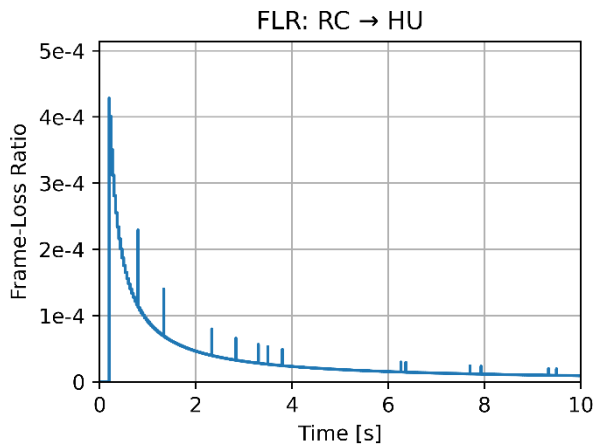


Figura 2: Andamento FLR: RC → HU

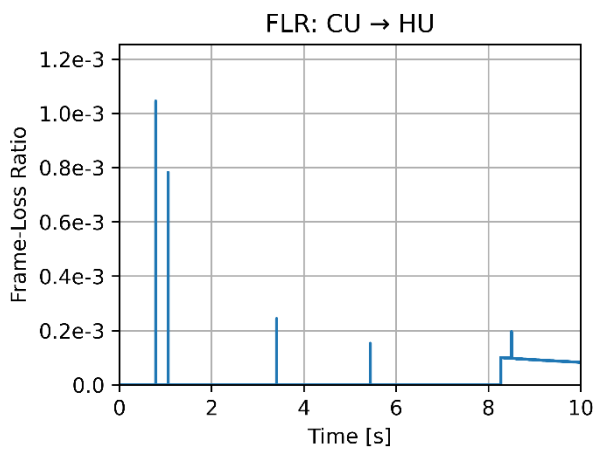


Figura 3: Andamento FLR: CU → HU

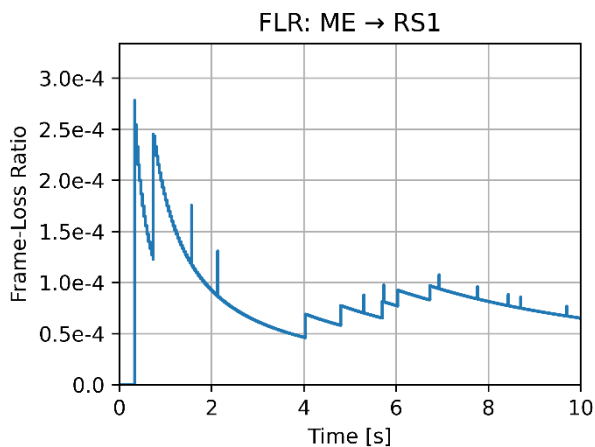


Figura 4: Andamento FLR: ME → RS1

- tendono a presentare **valori di FLR più elevati** all'insorgere delle perdite;
- mostrano una **diminuzione più lenta** della FLR rispetto ai flussi ridondati. Questo aspetto emerge chiaramente dal **confronto** tra **ME → RS1** (ridondato) e **ME → RS2** (*Without Redundancy*), che condividono caratteristiche analoghe a eccezione della destinazione: il flusso ridondato evidenzia una migliore capacità di **assorbire** le perdite e **stabilizzarsi** su valori inferiori di FLR nello stesso intervallo temporale.

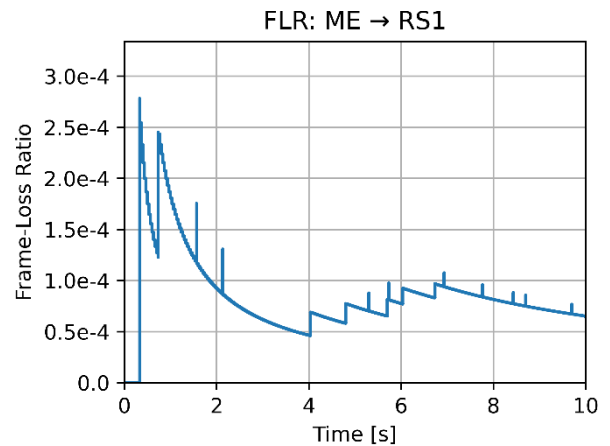


Figura 5: Andamento FLR: ME → RS1

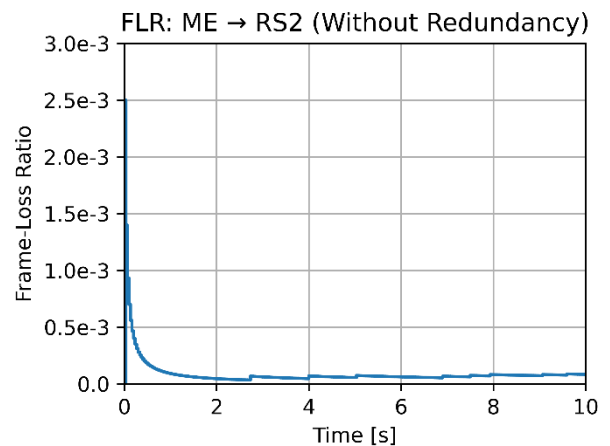


Figura 6: Andamento FLR: ME → RS2 (*Without Redundancy*)

### Flussi non ridondati

Per i flussi **senza ridondanza** (per scelta implementativa, quando sorgente e destinatario insistono sullo **stesso switch**), il comportamento qualitativo resta analogo—picchi in corrispondenza delle perdite seguiti da una riduzione della FLR—ma, in assenza di copie alternative del traffico, tali flussi risultano **più esposti** all'effetto della BER:

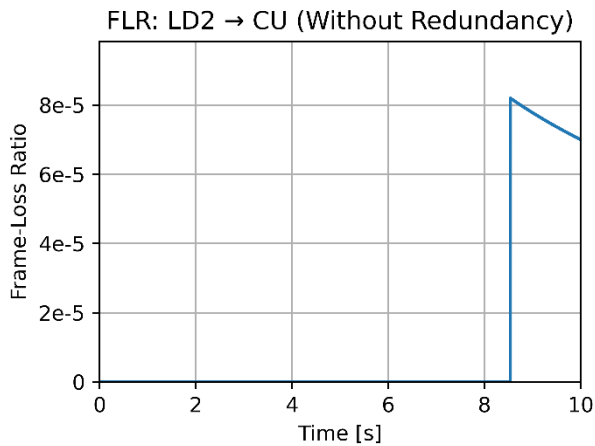


Figura 7: Andamento FLR: LD2 → CU (Without Redundancy)

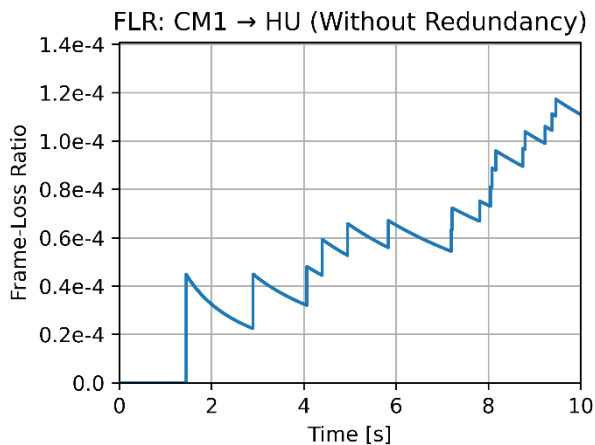


Figura 8: Andamento FLR: CM1 → HU (Without Redundancy)

### Sintesi

Nell'orizzonte di 10 s, l'assenza di BER rende nulla la FLR; con **BER**  $10^{-8}$  la perdita si manifesta secondo un andamento **impulsivo-transitorio** e la **ridondanza** dei percorsi riduce sia l'entità dei picchi sia il tempo di rientro verso valori bassi del rapporto di perdita.

### 2) Massimo numero di frame in coda di trasmissione (txQueue)

#### Invarianza rispetto alla BER

La statistica del **massimo numero di frame nella coda di trasmissione (txQueue)** non mostra differenze apprezzabili tra i due casi di canale considerati (assenza di BER vs  $BER = 10^{-8}$ ). Di conseguenza, l'analisi di questa metrica può prescindere dalla presenza di errori di bit.

#### Comportamento lato host trasmettenti

Dall'osservazione degli host **CU**, **CM1**, **RC** e **ME** emerge una relazione **quasi proporzionale** tra il **massimo occupamento di txQueue** e il **burst size** del flusso (ovvero il numero di frammenti che l'host genera periodicamente):

- **CU → HU**:  $burstSize = 12$  → il picco della txQueue è coerente con l'emissione simultanea dei 12 frammenti.
- **CM1 → HU**:  $burstSize = 255$  → si osserva un picco sensibilmente più elevato, in linea con il maggior numero di frammenti per periodo.
- **RC → HU**:  $burstSize = 356$  → il massimo di coda cresce ulteriormente, riflettendo l'incremento del burst.
- **ME** (caso particolare): genera più flussi in parallelo.
  - **ME → S1,S2,S3,S4**:  $burstSize = 1$  per ciascun flusso audio;
  - **ME → RS1, RS2**:  $burstSize = 357$  per ciascun flusso video.  
La **somma** dei due flussi verso RS1/RS2 è  $357 + 357 = 714$ , cui si aggiungono le emissioni audio (quattro burst da 1), per un totale **718** frammenti potenzialmente coesistenti in txQueue; il valore **osservato** di picco (**717**) risulta **molto vicino** a tale limite, confermando la dipendenza dell'occupazione massima dalla **sovrapposizione temporale** dei burst.

### Comportamento lato host riceventi e porte degli switch

Per gli host **riceventi** (più nello specifico, per le **porte degli switch** a cui sono collegati), l'occupazione massima della txQueue **non** raggiunge il valore del **burst size** del corrispondente flusso in ingresso. Ad esempio, sulla porta **SW3[4]** (collegata a **RS1**) la coda non arriva ai **357** attesi se si considerasse un accumulo istantaneo di tutti i frammenti del burst. Ciò è attribuibile a due fattori:

- **desincronizzazione** parziale tra le sorgenti, che riduce la probabilità di arrivi perfettamente allineati;
- **scheduling a Strict Priority Selection**, che lascia passare più rapidamente i frame ad alta priorità, impedendo alla coda di accumulare tutti i frammenti del burst.

### Porte di dorsale degli switch

Il **massimo numero di frame** in coda nelle porte **[0]** e **[1]** degli switch—ovvero le porte di collegamento agli **switch adiacenti**—dipende dal **numero di flussi instradati** su questi collegamenti e da **effetti di sincronizzazione** tra le sorgenti. I valori risultano **significativi** poiché queste porte costituiscono i **collegamenti di transito** principali della rete: la loro

occupazione di coda riflette la **convergenza** di molteplici flussi e l'efficacia dello **scheduling** nel contenere i picchi.

| Modulo                       | Value |
|------------------------------|-------|
| Scenario2.sw1.eth[0]         | 4     |
| Scenario2.sw1.eth[1]         | 1     |
| Scenario2.sw1.eth[2]         | 0     |
| Scenario2.sw1.eth[3]         | 0     |
| Scenario2.sw1.eth[4]         | 1     |
| Scenario2.sw1.eth[5]         | 532   |
| Scenario2.sw1.eth[6]         | 0     |
| Scenario2.sw1.eth[7]         | 0     |
| Scenario2.sw2.eth[0]         | 19    |
| Scenario2.sw2.eth[1]         | 7     |
| Scenario2.sw2.eth[2]         | 0     |
| Scenario2.sw2.eth[3]         | 4     |
| Scenario2.sw2.eth[4]         | 0     |
| Scenario2.sw2.eth[5]         | 1     |
| Scenario2.sw3.eth[0]         | 21    |
| Scenario2.sw3.eth[1]         | 6     |
| Scenario2.sw3.eth[2]         | 0     |
| Scenario2.sw3.eth[3]         | 1     |
| Scenario2.sw3.eth[4]         | 180   |
| Scenario2.sw3.eth[5]         | 0     |
| Scenario2.sw4.eth[0]         | 7     |
| Scenario2.sw4.eth[1]         | 358   |
| Scenario2.sw4.eth[2]         | 0     |
| Scenario2.sw4.eth[3]         | 0     |
| Scenario2.sw4.eth[4]         | 1     |
| Scenario2.sw4.eth[5]         | 1     |
| Scenario2.ld1.controller.mac | 1     |
| Scenario2.ld2.controller.mac | 1     |
| Scenario2.cu.controller.mac  | 11    |
| Scenario2.us1.controller.mac | 1     |
| Scenario2.us2.controller.mac | 1     |
| Scenario2.us3.controller.mac | 1     |
| Scenario2.us4.controller.mac | 1     |
| Scenario2.me.controller.mac  | 717   |
| Scenario2.s1.controller.mac  | 0     |
| Scenario2.s2.controller.mac  | 0     |
| Scenario2.s3.controller.mac  | 0     |
| Scenario2.s4.controller.mac  | 0     |
| Scenario2.hu.controller.mac  | 0     |
| Scenario2.cm1.controller.mac | 254   |
| Scenario2.rs1.controller.mac | 0     |
| Scenario2.rs2.controller.mac | 0     |
| Scenario2.rc.controller.mac  | 355   |
| Scenario2.tlm.controller.mac | 1     |

Tabella 3: Massimo numero di pacchetti nelle code di trasmissione

### 3) End-to-End delay (E2E Delay)

#### Invarianza rispetto alla BER

Coerentemente con quanto osservato per le altre metriche, l'**E2E delay** non mostra differenze apprezzabili tra i due casi di canale considerati (assenza di BER vs **BER** =  $10^{-8}$ ). Tale conclusione vale sia per il **ritardo dei singoli frame** sia per il **ritardo del burst**.

#### Flussi ME → S1, S2, S3, S4 (priorità più alta, **burstSize** = 1)

Per questi flussi audio ad **alta priorità**, il valore di **burstSize** = 1 implica che **E2E(frame)** = **E2E(burst)**. Le curve evidenziano un **livello base** dell'E2E delay nell'**ordine dei microsecondi**, con **picchi periodici** nettamente distinguibili (si vedano le figure "E2E Frame/Burst Delay: ME → S1...S4"). Tali picchi, con **periodicità**  $\approx 33,33$  ms, sono riconducibili all'**interferenza periodica** generata da flussi video ad alto volume di traffico (ME → RS1, RS2 e RC → HU), che provoca **accumuli transitori** nelle code di trasmissione nonostante la sua priorità inferiore. La **ridotta dimensione del payload** (80 B) mantiene comunque contenuto il tempo di trasmissione, permettendo all'E2E delay di **rientrare rapidamente** verso il livello base e di **rispettare la deadline** nell'intero orizzonte di simulazione.

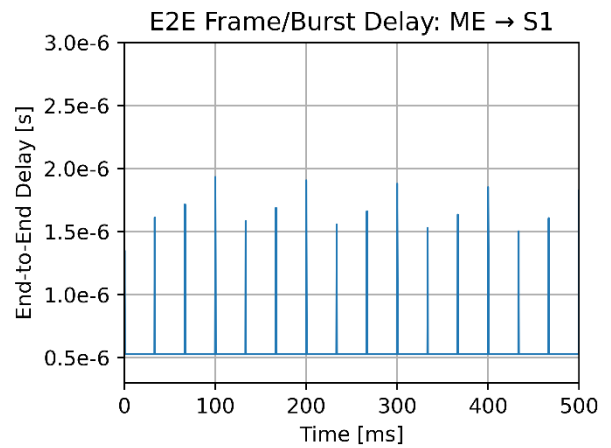


Figura 9: Andamento E2E Frame/Burst Delay: ME → S1

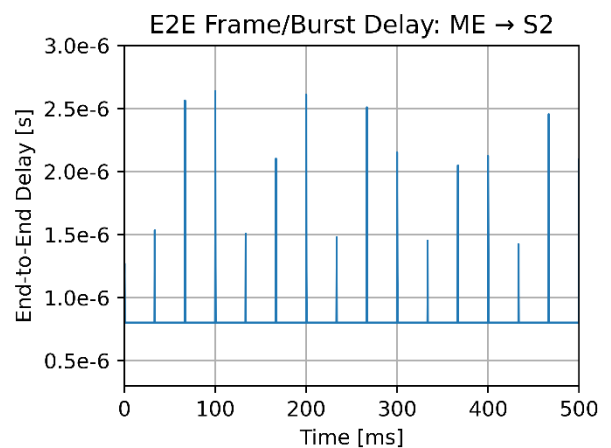


Figura 10: Andamento E2E Frame/Burst Delay: ME → S2

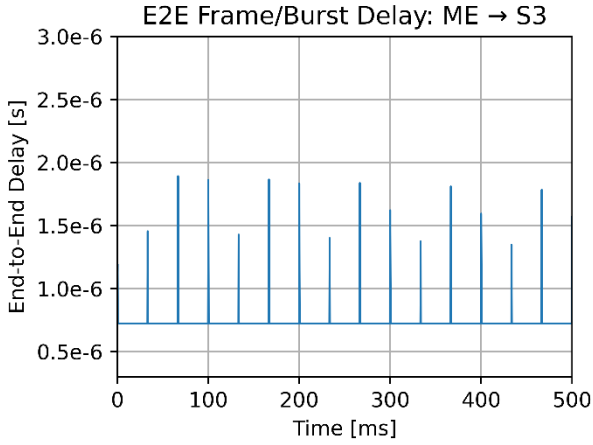


Figura 11: Andamento E2E Frame/Burst Delay: ME → S3

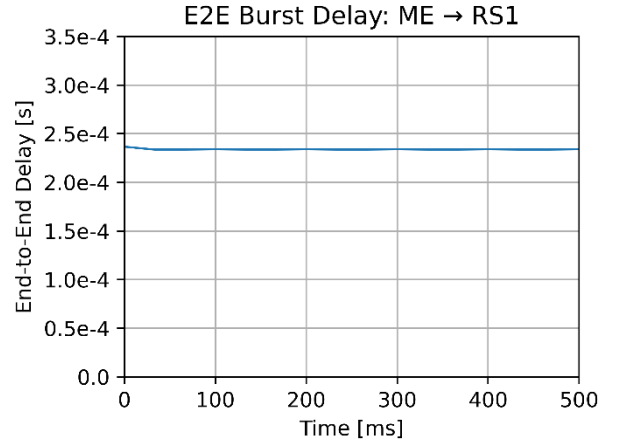


Figura 13: Andamento E2E Delay: ME → RS1

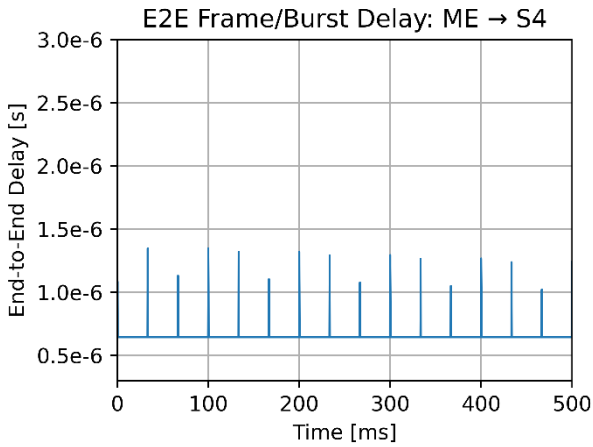


Figura 12: Andamento E2E Frame/Burst Delay: ME → S4

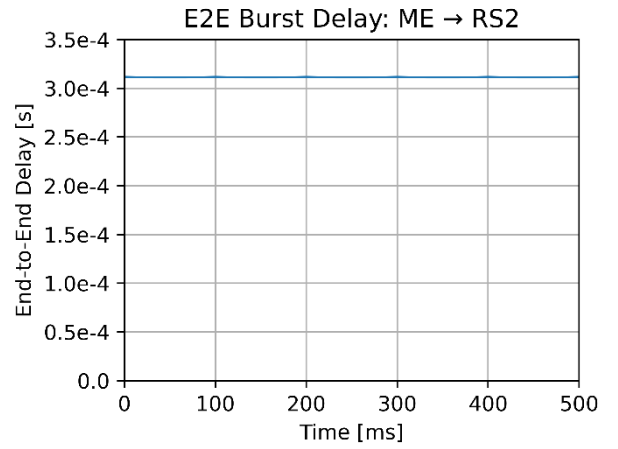


Figura 14: Andamento E2E Delay: ME → RS2

#### Flussi ME → RS1, RS2 (priorità intermedia)

Per i due flussi video diretti ai dispositivi **Rear Seat**, le tracce dell'E2E delay non mostrano **componenti dinamiche rilevanti** imputabili all'interferenza di altri flussi, pur trattandosi di traffico a **priorità 2**. Li presentiamo tuttavia per sottolineare che un minor numero di collegamenti attraversati non si traduce necessariamente in un E2E delay inferiore, come dimostrato da questi flussi. Questa situazione anomala è quasi certamente dovuta a **fenomeni di congestione interna**, dato che ME è responsabile anche della generazione — molto più frequente — dei pacchetti destinati alle sorgenti sonore S1, S2, S3, S4.

#### Flussi US1, US2, US3, US4 → CU (priorità più bassa, *burstSize = 1*)

I flussi dei **sensori ad ultrasuoni** presentano anch'essi *burstSize = 1*, per cui **E2E(frame) = E2E(burst)**. Le curve mostrano un **transitorio iniziale** (picchi isolati o rampe iniziali) dovuto all'**assestamento** delle code nelle prime fasi della simulazione e all'**interazione** con burst provenienti da flussi a priorità superiore. Dopo tali transitori, l'E2E delay si stabilizza su valori **bassi e costanti** (ordine del microsecondo), grazie alle ridotte **dimensione delle frame** e all'efficacia dello **scheduling** nel drenare prima i traffici più critici.



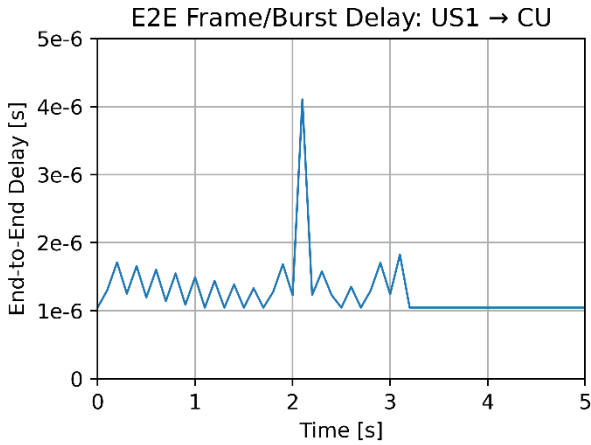


Figura 15: Andamento E2E Frame/Burst Delay: US1 → CU

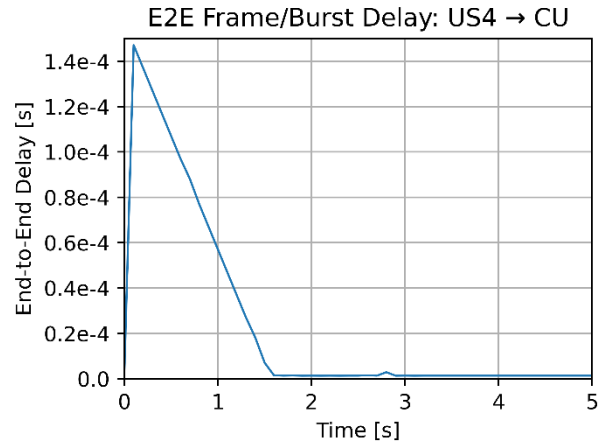


Figura 18: Andamento E2E Frame/Burst Delay: US4 → CU

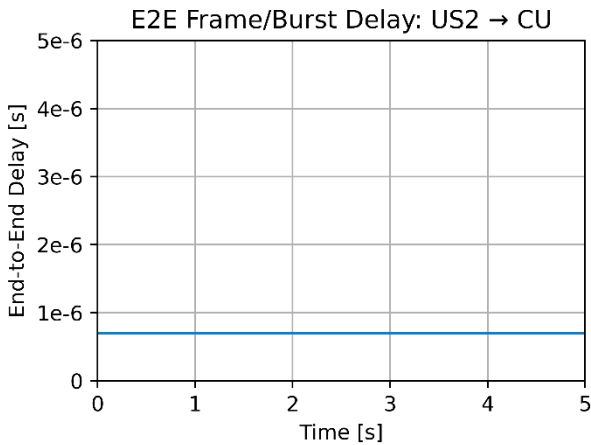


Figura 16: Andamento E2E Frame/Burst Delay: US2 → CU

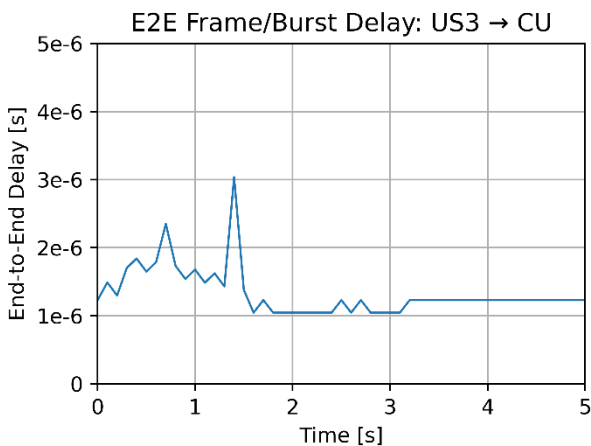


Figura 17: Andamento E2E Frame/Burst Delay: US3 → CU

#### Flusso US2 → CU (baseline).

L'andamento dell'E2E delay è **costante** e si attesta sui valori **più bassi** tra i flussi degli ultrasuoni. La stabilità è imputabile al fatto che **US2** e **CU** sono connessi allo **stesso switch**: il numero di collegamenti attraversati è minimo e l'assenza di tratte di dorsale riduce l'esposizione a congestione.

#### Flussi US1 → CU e US3 → CU.

Pur mantenendosi su valori **contenuti**, l'E2E delay mostra una **variabilità** più marcata rispetto a US2 → CU. La causa principale è il **traffico intenso** sui collegamenti **SW1→SW2** e **SW3→SW2**; in presenza di **Strict Priority Selection**, i burst dei flussi più prioritari inducono piccoli **transienti** (picchi o dentellature) che spiegano l'andamento osservato.

#### Flusso US4 → CU.

Il **picco iniziale** è riconducibile alla combinazione tra **maggiore numero di collegamenti** attraversati (il più elevato tra i quattro flussi US\*) e gli **effetti di scheduling** basati su priorità. Dopo il transitorio, il ritardo si **assesta** rapidamente su valori bassi.

#### Verifica delle deadline (US\* → CU).

In tutti e quattro i casi l'E2E delay rimane **sufficientemente basso** da **rispettare le deadline** nell'intero intervallo di simulazione considerato.

#### Pattern ricorrenti su altri flussi (LD1/LD2 → CU, CU → HU, CM1 → HU).

Nei flussi indicati si osservano **comportamenti analoghi** a quelli già descritti: lievi **oscillazioni periodiche** o **transienti puntuali** dovuti all'interazione con traffico concorrente a priorità più alta e alla **frammentazione**. Le figure dedicate mostrano, ad esempio, il **profilo a dente di sega** per CM1 → HU e i **picchi isolati** per LD1/LD2 → CU (compatibili con brevi congestioni sulle porte di dorsale).

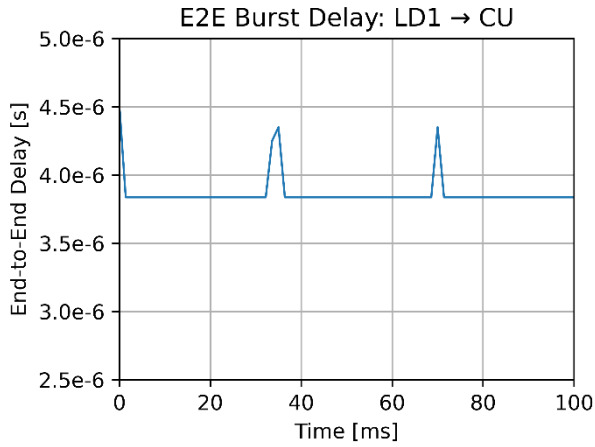


Figura 19: Andamento E2E Burst Delay: LD1 → CU

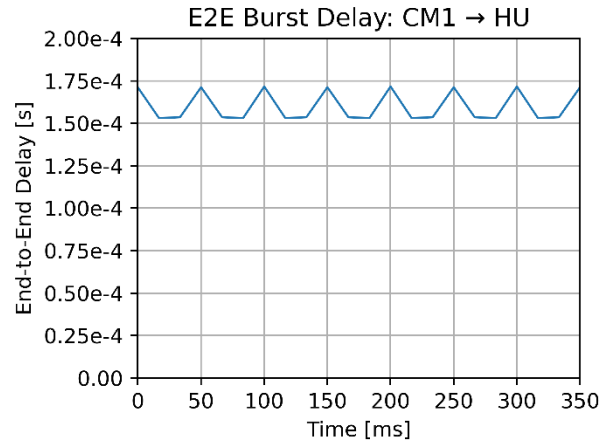


Figura 22: Andamento E2E Burst Delay: CM1 → HU

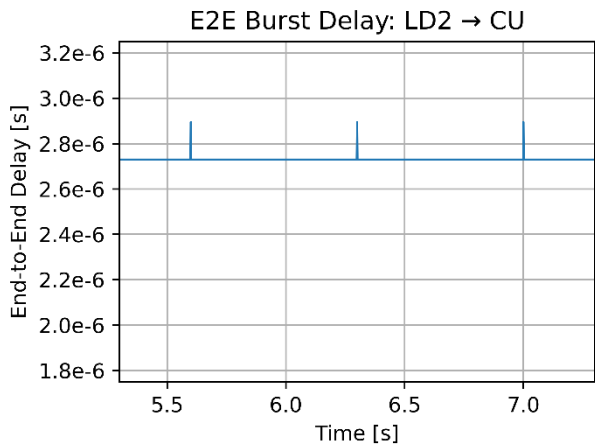


Figura 20: Andamento E2E Burst Delay: LD2 → CU

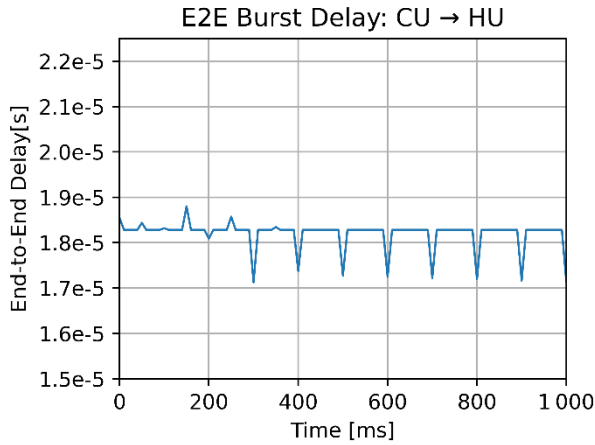


Figura 21: Andamento E2E Burst Delay: CU → HU

### Corrispondenza temporale dei picchi tra E2E frame e E2E burst (LD1 → CU)

Confrontando il tracciato dell'E2E delay a livello di **frame** per LD1 → CU (Fig. 19) con il corrispondente profilo a livello di **burst** (Fig. 23), si osserva una **perfetta corrispondenza temporale dei picchi**: gli istanti in cui compaiono le escursioni del ritardo risultano **identici** in entrambi i grafici.

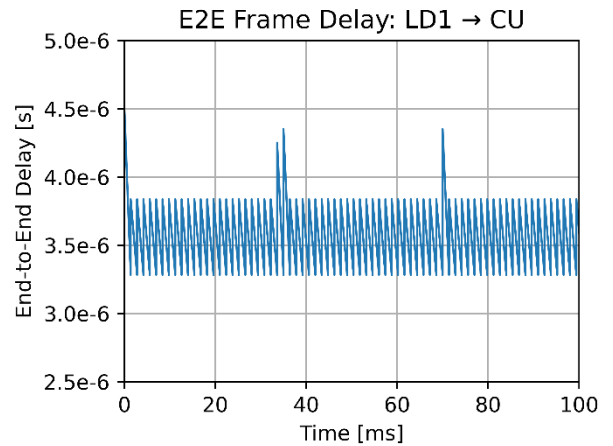


Figura 23: Andamento E2E Frame Delay: LD1 → CU

### Flussi rimanenti (TLM → CU, TLM → HU, RC → HU).

Gli andamenti registrati sono **riconducibili** ai casi precedenti o a **combinazioni** dei medesimi meccanismi (topologia più/meno profonda, interferenza periodica, priorità, frammentazione). Per completezza si riportano i grafici senza ulteriori commenti.

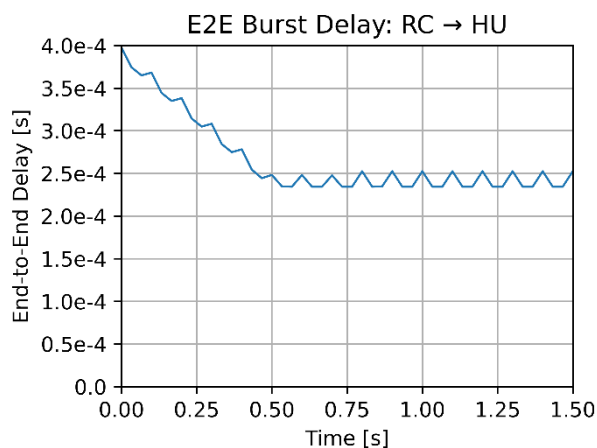


Figura 24: Andamento E2E Frame Delay: RC → HU

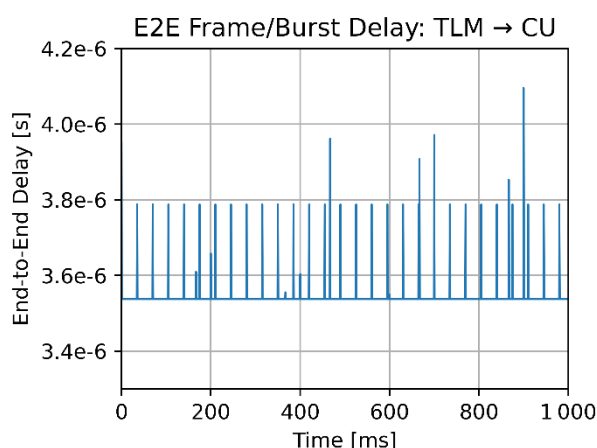


Figura 25: Andamento E2E Frame/Burst Delay: TLM → CU

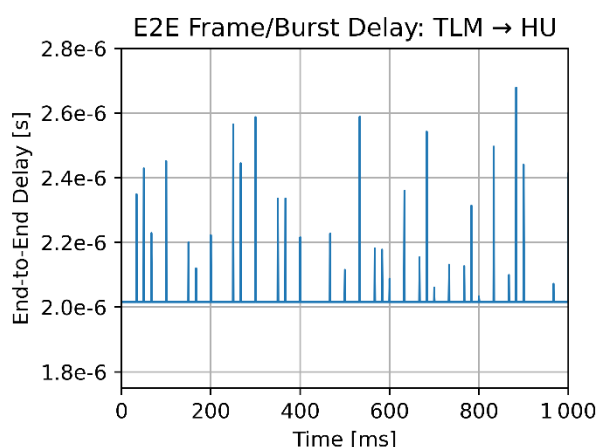


Figura 26: Andamento E2E Frame/ Burst Delay: TLM → HU

## Sintesi

Su tutti i flussi analizzati, l'E2E delay mostra:

- **assenza di sensibilità** alla BER nell'intervallo considerato;
- **picchi periodici** legati a interferenze regolari dei flussi video ad alto volume;

- **rapida riconvergenza** al livello base grazie alla combinazione di **frammentazione e Strict Priority Selection**;
- **rispetto delle deadline** in tutti i casi osservati nell'orizzonte di simulazione.

## Osservazione finale.

In tutti i flussi analizzati le **deadline risultano rispettate nel periodo di 10 s** simulato. È importante sottolineare tuttavia che il rispetto delle deadline su questo orizzonte temporale **non costituisce una garanzia assoluta**, in quanto variazioni dei parametri di configurazione (come dimensioni delle code, priorità dei flussi o burst size) potrebbero comportare violazioni delle deadline anche in condizioni di carico simili.